

TRAVEXA - SMART TRAVEL PLANNER

A Mini Project Report

Submitted in partial fulfillment of requirement of the

Degree of

**BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE &
ENGINEERING**

BY

Aditi Patidar	Abhisar Joshi
EN23CS301059	EN23CS301045

Under the Guidance of
Prof. Ashish Kumar Kumawat



Department of Computer Science & Engineering
Faculty of Engineering
MEDICAPS UNIVERSITY, INDORE- 453331

APRIL-2026

Report Approval

The project work "**TRAVEXA - SMART TRAVEL PLANNER**" is hereby approved as a creditable study of an engineering/computer application subject carried out and presented in a manner satisfactory to warrant its acceptance as prerequisite for the Degree for which it has been submitted.

It is to be understood that by this approval the undersigned do not endorse or approve any statement made, opinion expressed, or conclusion drawn therein; but approve the "Project Report" only for the purpose for which it has been submitted.

Internal Examiner
Prof. Ashish Kumawat
Assistant Professor
Medicaps University

Declaration

We hereby declare that the project entitled "**TRAVEXA - SMART TRAVEL PLANNER**" submitted in partial fulfillment for the award of the degree of Bachelor of Technology in Computer Science and Engineering, completed under the supervision of **Prof. Ashish Kumawat, Assistant Professor, Computer Science and Engineering**, Faculty of Engineering, Mediacaps University Indore, is an authentic work.

Further, we declare that the content of this Project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for the award of any degree or diploma.

Signature and name of the student(s) with date

Certificate

I, **Prof. Ashish Kumawat** certify that the project entitled "**TRAVEXA - SMART TRAVEL PLANNER**" submitted in partial fulfillment for the award of the degree of Bachelor of Technology by **Aditi Patidar and Abhisar Joshi** is the record carried out by them under my guidance and that the work has not formed the basis of award of any other degree elsewhere.

Prof. Ashish Kumawat

Department of Computer Science and Engineering

Medicaps University, Indore

Dr. Kailash Chandra Bandhu

Head of the Department

Computer Science & Engineering

Medicaps University, Indore

Acknowledgements

We would like to express our deepest gratitude to the Honorable Chancellor, **Shri R C Mittal**, who has provided us with every facility to successfully carry out this project, and our profound indebtedness to **Prof. (Dr.) D. K. Patnaik**, Vice Chancellor, Medicaps University, whose unfailing support and enthusiasm has always boosted up our morale. We also thank **Prof. (Dr.) Ratnesh Litoriya**, Associate Dean, Faculty of Computing Engineering, Medicaps University, for giving us a chance to work on this project. We would also like to thank our Head of the Department **Dr. Kailash Chandra Bandhu** for his continuous encouragement for the betterment of the project.

It is their help and support, due to which we became able to complete the design and technical report.

Without their support this report would not have been possible.

Aditi Patidar

Abhisar Joshi

B.Tech. III Year (6A)

Department of Computer Science & Engineering

Faculty of Engineering

Medicaps University, Indore

Abstract

TRAVEXA is a web-based Smart Travel Planning and Booking System developed using the Python Flask framework and MySQL database. The system provides a unified digital platform for users to search, view, and book transportation services including flights, trains, and buses from a single interface. The application integrates the Amadeus API for real-time flight data and RapidAPI for live train schedules, while bus data is simulated using a realistic mock data engine.

The system implements a complete end-to-end travel workflow covering user registration and session-based authentication, service search by source and destination, ticket booking with passenger detail collection, payment processing via an integrated E-Wallet and simulated card/UPI/net banking options, one-click booking cancellation, and automated refund crediting to the user's E-Wallet. Refund logic follows real-world business rules — full refund within 24 hours of booking, 80% refund for cancellations made more than two hours before departure, and no refund for last-minute cancellations.

The backend is built using Flask routes, session management, and MySQL CRUD operations. The frontend uses HTML templates and CSS styling. The system demonstrates the practical application of client-server architecture, relational database design, API integration, and session management in an academic context.

Keywords : Smart Travel Planner, Flask, MySQL, Amadeus API, RapidAPI, E-Wallet, Booking System, Session Management, Python, REST API.

Table of Contents

		Page No.
	Report Approval	ii
	Declaration	iii
	Certificate	iv
	Acknowledgement	v
	Abstract	vi
	Table of Contents	vii
	List of Figures	ix
	Abbreviations	x
	Notations & Symbols	xi
Chapter 1	Introduction	
	1.1 Introduction	1
	1.2 Literature Review	1
	1.3 Objectives	2
	1.4 Significance	3
	1.5 Chapter Scheme	3
Chapter 2	Requirements Specification	
	2.1 User Characteristics	4
	2.2 Functional Requirements	4
	2.3 Dependencies	5
	2.4 Performance Requirements	5
	2.5 Hardware Requirements	6
	2.6 Constraints & Assumptions	6
Chapter 3	Design	
	3.1 Algorithm	8
	3.2 Function Oriented Design	9
	3.3 System Design	9
	3.3.1 Data Flow Diagrams (Level 0, Level 1)	10

	3.3.2 Use Case Diagram	11
	3.3.3 Class Diagram	12
	3.4 Database Design	13
	3.4.1 Logical Database Design	14
Chapter 4	Implementation, Testing, and Maintenance	
	4.1 Introduction to Languages, IDEs, Tools and Technologies	16
	4.2 Testing Techniques and Test Plans	17
	4.3 Installation Instructions	19
	4.4 End User Instructions	20
Chapter 5	Results and Discussions	
	5.1 User Interface Representation	21
	5.2 Brief Description of Various Modules	21
	5.3 Snapshots of System with Brief Detail	22
	5.4 Back End Representation	26
	5.5 Snapshots of Database Tables	28
Chapter 6	Summary and Conclusions	30
Chapter 7	Future Scope	31
	Appendix	32
	Bibliography	33

List of Figures

Figure No.	Title	Page No.
Figure 3.1	Level 0 Data Flow Diagram (Context Diagram)	10
Figure 3.2	Level 1 Data Flow Diagram	10
Figure 3.3	Use Case Diagram	11
Figure 3.6	Class Diagram	12
Figure 5.1	Login / Signup Screen	22
Figure 5.2	Home Screen — Service Selection	23
Figure 5.3	Flight Search Results Screen	23
Figure 5.4	Train Search Results Screen	24
Figure 5.5	Bus Search Results Screen	24
Figure 5.6	Passenger Detail Entry Screen	25
Figure 5.7	Payment Screen	25
Figure 5.8	My Bookings Screen	26
Figure 5.9	E-Wallet Screen	26
Figure 5.10	Database Tables — MySQL Workbench View	29

Abbreviations

Abbreviation	Full Form
API	Application Programming Interface
CSS	Cascading Style Sheets
CRUD	Create, Read, Update, Delete
DB	Database
DFD	Data Flow Diagram
ER	Entity-Relationship
FK	Foreign Key
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol
IDE	Integrated Development Environment
IATA	International Air Transport Association
JSON	JavaScript Object Notation
MVC	Model-View-Controller
MySQL	My Structured Query Language
PC	Personal Computer
PK	Primary Key
REST	Representational State Transfer
SQL	Structured Query Language
UI	User Interface
URL	Uniform Resource Locator
UML	Unified Modeling Language
UPI	Unified Payments Interface

Notations & Symbols

Notation / Symbol	Meaning
<code>session['user']</code>	Flask session key storing logged-in username
GET / POST	HTTP request methods used by Flask routes
<code>@app.route(...)</code>	Flask decorator defining URL endpoints
<code>cursor.execute(...)</code>	MySQL query execution method
<code>Decimal(...)</code>	Python decimal type used for precise price calculations
<code>refund_amount</code>	Computed refund value based on cancellation rules
<code>balance</code>	E-Wallet balance stored in wallet table
PK	Primary Key — unique row identifier in a table
FK	Foreign Key — references primary key of another table
%	Wildcard operator in SQL LIKE queries
→	Maps to / redirects to (in data flow and route diagrams)
*	One-or-more relationship multiplicity

Chapter 1

Introduction

1.1 Introduction

The rapid advancement of digital technologies has transformed the way people plan and manage their travel. Online booking systems have replaced traditional manual reservation systems due to their speed, convenience, and accessibility. Customers today expect seamless booking experiences for flights, trains, and buses through centralized digital platforms.

The proposed project, TRAVEXA — Smart Travel Planner, is a web-based application designed to provide users with an integrated platform for searching, viewing, and booking travel services across three major modes of transportation: flights, trains, and buses. The system integrates live flight data through the Amadeus API, real-time train schedules through the RapidAPI IRCTC endpoint, and a realistic bus data simulation engine that models demand-based pricing and availability.

The application demonstrates real-world booking system concepts including user authentication, API integration, session management, database-driven booking and wallet management, and automated refund processing — all within an academically feasible implementation using the Python Flask framework and MySQL.

1.2 Literature Review

Various online travel platforms exist in the current market. A review of existing systems and academic models reveals the following:

Online Flight Reservation Systems: Platforms such as MakeMyTrip and IRCTC Air allow users to search, compare, and book flight tickets using live airline inventory APIs. These systems incorporate schedule viewing, seat availability, dynamic pricing, cancellation policies, and digital payment integration[1][2].

Bus Booking Portals: Services such as RedBus and AbhiBus enable passengers to select routes, view available buses, choose seats, and complete ticket reservations online, making the booking process fast and convenient[3][4].

Railway Reservation Systems: IRCTC and similar platforms manage train schedules, seat availability, ticket booking, cancellation, and refund processing through centralized digital platforms[5]

Integrated Travel Portals: Platforms such as Cleartrip and Yatra combine multiple transportation services into a single interface, allowing users to plan their complete journey from one place[6][7]

Despite their efficiency, such platforms are complex and large-scale in nature. They require heavy infrastructure, cloud deployment, and integration with multiple third-party APIs at commercial scale. Research and existing academic models indicate that web-based reservation systems improve accessibility, database-driven systems ensure structured data management, and session-based authentication enhances security[8]. The proposed project adopts core functionalities of modern travel systems while simplifying the overall design to practical academic implementation constraints.

References

- [1] MakeMyTrip Official Website: <https://www.makemytrip.com>
- [2] IRCTC Air: <https://www.air.irctc.co.in>
- [3] RedBus: <https://www.redbus.in>
- [4] AbhiBus: <https://www.abhibus.com>
- [5] IRCTC Rail Connect: <https://www.irctc.co.in>
- [6] Cleartrip: <https://www.cleartrip.com>
- [7] Yatra: <https://www.yatra.com>
- [8] IRCTC Overview (Wikipedia): https://en.wikipedia.org/wiki/Indian_Railway_Catering_and_Tourism_Corporation

1.3 Objectives

The primary objectives of the TRAVEXA project are as follows:

1.3.1 Functional Objectives

- To develop a secure Login and Signup system with session-based authentication and server-side validation.
- To design a Home Page with three service cards for Flights, Trains, and Buses.
- To implement search functionality for flights, trains, and buses by source and destination.

- To simulate and implement the booking process with customer information collection.
- To enable one-click booking cancellation with automated refund rules.
- To implement an E-Wallet refund mechanism that automatically credits cancellation refunds.

1.3.2 Technical Objectives

- To use Flask for backend development with route-based request handling.
- To integrate MySQL database for user storage, booking records, and wallet management.
- To implement session management for maintaining user login state.
- To integrate the Amadeus API for live flight data and RapidAPI for live train data.

1.3.3 Educational Objectives

- To understand client-server architecture through practical implementation.
- To implement CRUD operations on a relational MySQL database.
- To demonstrate database connectivity between Flask and MySQL.
- To learn safe form data handling and session-based authentication.

1.4 Significance

TRAVEXA holds practical significance in the academic context as it demonstrates the complete lifecycle of a real-world web application — from user registration and API integration to payment processing and refund management — within the constraints of an academic mini project. For students, the system provides hands-on exposure to client-server architecture, database integration, backend logic building, and session management. For evaluators, the project demonstrates how modern software engineering concepts including REST API consumption, automated refund logic, and session-based authentication can be implemented in a cohesive and user-friendly application.

1.5 Chapter Scheme

The remainder of this report is organized as follows. Chapter 2 describes the requirements specification covering user characteristics, functional requirements, dependencies, performance requirements, hardware requirements, and constraints. Chapter 3 presents the complete system design including data flow diagrams, use case diagram, activity diagram,

flowchart, class diagram, ER diagram, and database design. Chapter 4 covers implementation details including the technology stack, test cases, installation instructions, and end-user operating instructions. Chapter 5 presents results and discussions including user interface descriptions, module descriptions, and database representation. Chapter 6 provides a summary and conclusions. Chapter 7 outlines the future scope of the system.

Chapter 2

Requirements Specification

2.1 User Characteristics

The TRAVEXA system is designed to serve a single user role — the Traveller. A Traveller is a registered user who accesses the system to search, book, and manage travel reservations. The user is expected to have basic familiarity with web browsers and online booking platforms. No technical background is required. The user registers using a username and password, logs into the system, and interacts with the interface to explore flight, train, or bus options and complete bookings.

2.2 Functional Requirements

The functional requirements of TRAVEXA are enumerated in Table 2.1 below. Each requirement is assigned a unique identifier and priority level.

Table 2.1: Functional Requirements

Req. ID	Requirement Description	Priority
FR-01	System shall allow new users to register with a unique username and password.	High
FR-02	System shall authenticate users via session-based login with server-side validation.	High
FR-03	System shall display a home page with three service cards: Flights, Trains, and Buses.	High
FR-04	System shall allow users to search flights by source (IATA code) and destination (IATA code) using the Amadeus API.	High
FR-05	System shall allow users to search trains by source station code and destination code using the RapidAPI IRCTC endpoint.	High
FR-06	System shall allow users to search buses by source city and destination city using the mock bus data engine.	High

FR-07	System shall display available travel options with details including operator name, departure time, arrival time, price, and status.	High
FR-08	System shall allow users to book a selected travel option by entering passenger details (name, age, gender, mobile, email).	High
FR-09	System shall process payment via E-Wallet balance deduction or simulated card/UPI/net banking.	High
FR-10	System shall generate a booking record in the database upon successful payment.	High
FR-11	System shall allow users to view all their bookings on the My Bookings screen.	Medium
FR-12	System shall allow users to cancel a confirmed booking with one click.	Medium
FR-13	System shall process refunds based on cancellation rules and credit the amount to the user's E-Wallet.	Medium
FR-14	System shall display the E-Wallet balance on the home page and wallet screen.	Medium
FR-15	System shall display an appropriate message when no travel options are found for a searched route.	Low

2.3 Dependencies

The following software dependencies are required for operation of the TRAVEXA system:

- Backend: Python 3.9 or higher, Flask 2.x, mysql-connector-python, amadeus SDK, requests library, and the python-decimal module.
- Database: MySQL Server 8.0 or higher. The travel_planner database must be created with the required tables before the application is started.
- External APIs: Amadeus for Developers account with valid client_id and client_secret for flight search. RapidAPI subscription to the IRCTC1 API for train data.
- Browser: Any modern browser supporting JavaScript (Chrome 90+, Firefox 88+) for the HTML/CSS frontend.
- Network: Active internet connection is required for Amadeus and RapidAPI calls. Bus data is generated locally without a network call.

2.4 Performance Requirements

The following performance requirements define the expected operational characteristics of the TRAVEXA system:

- The system shall respond to all local Flask GET requests within 500 milliseconds under standard single-user load.
- Flight search results shall be returned within 5 seconds, subject to Amadeus API response time.
- Train search results shall be returned within 8 seconds, subject to RapidAPI response time.
- Bus search results shall be returned within 3 seconds including the simulated 1.5-second artificial delay added for UX realism.
- All database transactions including booking, cancellation, and wallet updates shall complete within 1 second under standard conditions.

2.5 Hardware Requirements

Table 2.2: Hardware Requirements

Component	Specification	Minimum	Recommended
Server / Dev PC	Processor	Dual-core 1.8 GHz	Quad-core 2.5 GHz+
Server / Dev PC	RAM	4 GB	8 GB
Server / Dev PC	Storage	20 GB HDD	50 GB SSD
Server / Dev PC	OS	Windows 10 / Ubuntu 20.04	Windows 11 / Ubuntu 22.04
Network	Internet Bandwidth	10 Mbps	100 Mbps

2.6 Constraints & Assumptions

- API Rate Limits: The Amadeus API has daily call limits on the free tier. Once exhausted, flight search returns no results. The system does not currently cache API responses.

- Payment Simulation: Payment processing via card, UPI, and net banking is simulated with a 90% success rate. No actual financial transactions are performed.
- Bus Data: Bus search results are entirely mock-generated. No real bus operator data is used.
- Single-User Design: The system is designed for single-user sessions on a local development server. No multi-user concurrency control has been implemented.
- Database Pre-requisite: MySQL must be installed, running, and the travel_planner schema must be initialized before the Flask application is started.
- Authentication: Passwords are stored as plain text in the database. Password hashing has not been implemented in this academic version.

Chapter 3

Design

3.1 Algorithm

The core algorithms of TRAVEXA are the Booking Algorithm and the Refund Computation Algorithm, described below:

3.1.1 Booking Algorithm

Step 1 — Availability Check: The system queries the relevant API (Amadeus for flights, RapidAPI for trains) or generates mock data (buses). Results with past departure times are filtered out.

Step 2 — Service Selection: The user selects a travel option. Passenger details (name, age, gender, mobile, email) are collected via a form.

Step 3 — Fare Calculation: Total amount = base fare + convenience fee (Rs. 350 for flights). For trains and buses, the price from the search result is used directly.

Step 4 — Payment Processing: If E-Wallet is selected, the wallet balance is checked. If balance $\geq$ total amount, the amount is deducted. For card/UPI/net banking, a random success simulation is applied (90% success rate).

Step 5 — Booking Record Creation: On payment success, a booking record is inserted into the bookings table and a passenger record is inserted into the passengers table. A booking_status of 'Confirmed' and refund_status of 'Not Cancelled' are set.

3.1.2 Refund Computation Algorithm

Step 1: Retrieve the booking record by booking_id and username.

Step 2: Check if departure_time - now < 2 hours. If yes, refund_amount = 0, refund_status = 'No Refund'.

Step 3: Else if now - booking_time $\leq$ 24 hours, refund_amount = price (full refund), refund_status = 'Full Refund'.

Step 4: Else refund_amount = price * 0.8 (80% refund), refund_status = '80% Refunded'.

Step 5: Update `booking_status = 'Cancelled'` in the bookings table. If `refund_amount > 0`, add `refund_amount` to the user's wallet balance.

3.2 Function Oriented Design

The TRAVEXA backend follows a function-oriented design using Flask routes as the primary decomposition unit. Each Flask route corresponds to a single system function:

- `welcome()`: Renders the landing page with login and signup forms.
- `login()`: Validates credentials against the users table and creates a Flask session.
- `signup()`: Validates new user data, inserts into users and wallet tables, and creates a session.
- `home()`: Displays the home page with service cards and current wallet balance.
- `search_flights()`: Calls the Amadeus API, filters results by source/destination and departure time, and renders the results.
- `search_trains()`: Calls the RapidAPI IRCTC endpoint, processes real train data, and renders results.
- `search_buses()`: Generates mock bus data using `get_buses_api_data()` and renders results.
- `enter_passenger()`: Stores flight selection in session and renders the passenger detail form.
- `payment()`: Calculates total fare and renders the payment screen.
- `process_payment()`: Validates wallet balance or simulates card/UPI payment, inserts booking and passenger records.
- `my_bookings()`: Retrieves and displays all bookings for the logged-in user.
- `cancel_booking()`: Applies refund logic, updates booking status, and credits wallet.
- `wallet()`: Displays the current E-Wallet balance.
- `logout()`: Clears the Flask session and redirects to the login page.

3.3 System Design

3.3.1 Data Flow Diagrams (Level 0 and Level 1)

The Data Flow Diagrams (DFDs) model the movement of data through the TRAVEXA system at two levels of abstraction. Figure 3.1 shows the Level 0 (context) diagram, and Figure 3.2 shows the Level 1 diagram depicting internal process decomposition.

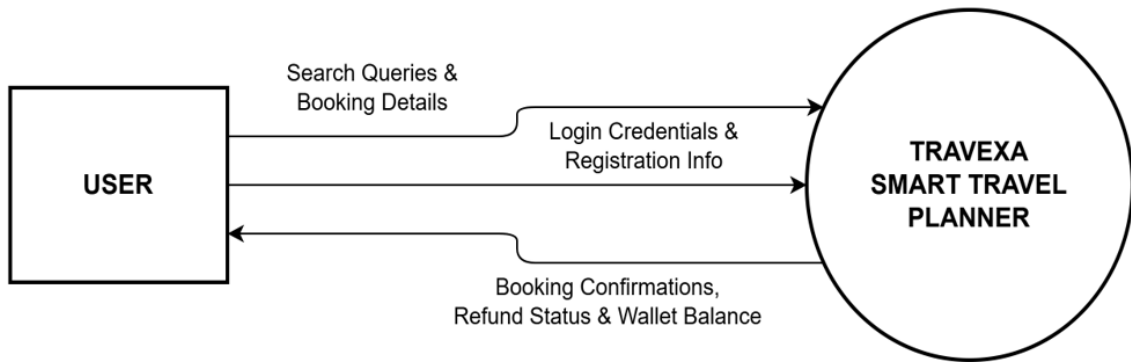


Figure 3.1: Level 0 Data Flow Diagram (Context Diagram)

At Level 0, two external entities interact with the TRAVEXA system: the User who registers, searches, books, and cancels, and the External APIs (Amadeus and RapidAPI) which supply live travel data. At Level 1, the system is decomposed into five processes: (1) User Authentication — handling registration and login; (2) Travel Search — querying APIs and generating search results; (3) Booking Management — processing passenger details and payment; (4) Cancellation & Refund — applying refund rules and updating wallet; and (5) Wallet Management — maintaining and displaying the E-Wallet balance.

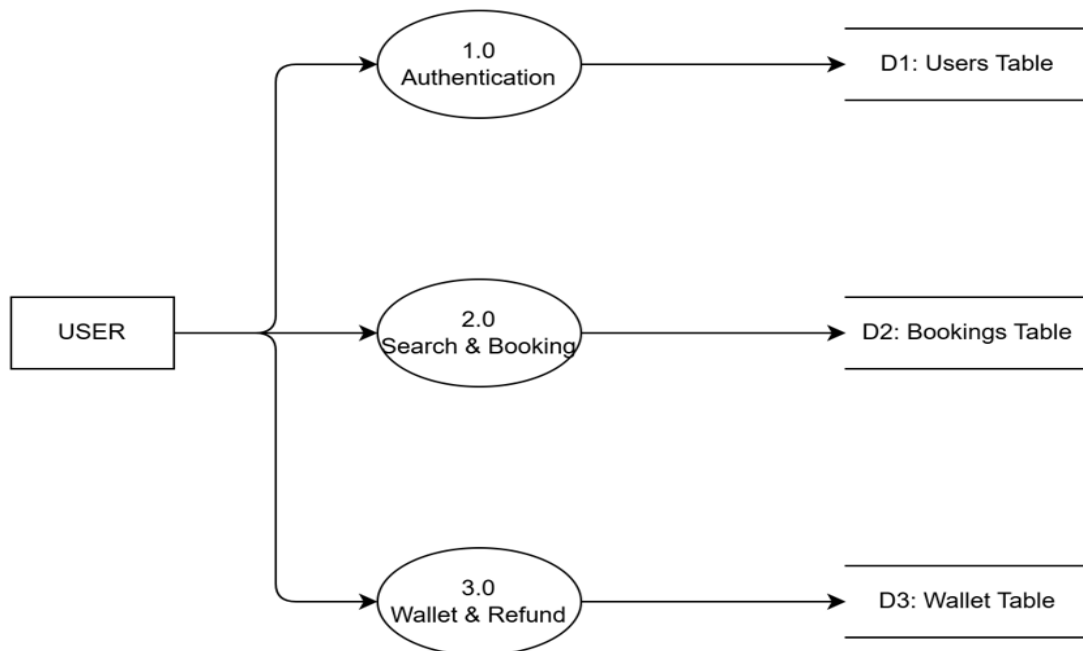


Figure 3.2: Level 1 Data Flow Diagram

3.3.2 Use Case Diagram

The Use Case Diagram models the interactions between the User (the single actor in the system) and the TRAVEXA system. Figure 3.3 shows the complete use case diagram. The primary use cases include: Register, Login, Search Flights, Search Trains, Search Buses, Book Ticket, Enter Passenger Details, Make Payment, View Bookings, Cancel Booking, View Wallet, and Logout.

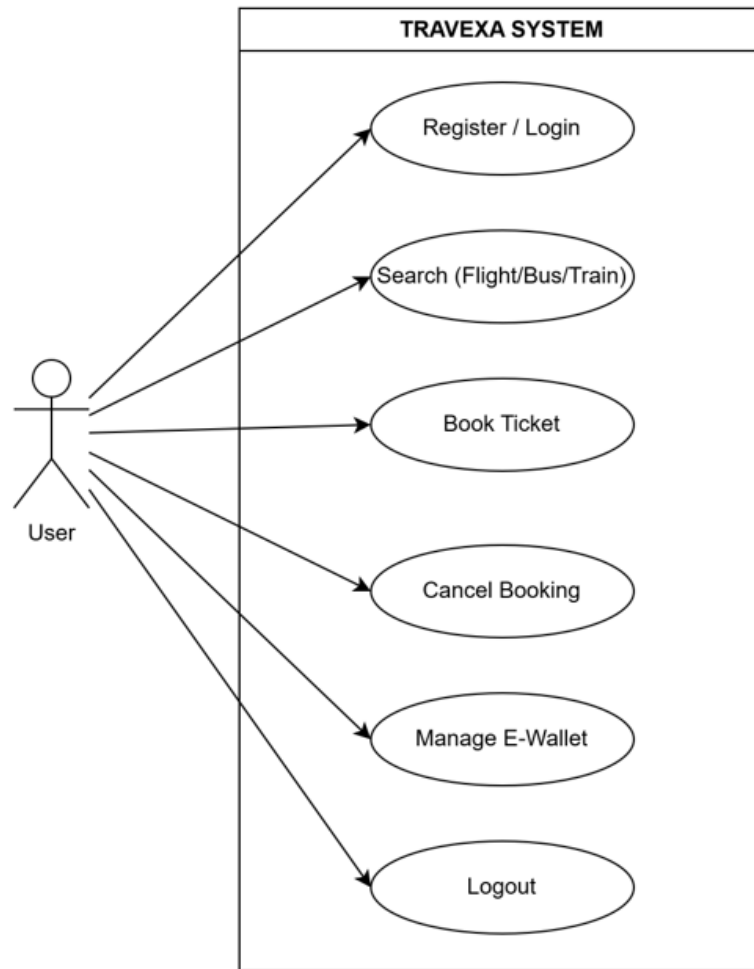


Figure 3.3: Use Case Diagram

3.3.3 Class Diagram

The Class Diagram represents the static structure of the TRAVEXA system, showing the logical classes, their attributes, methods, and relationships. The primary classes are: User (username,

password; methods: login, signup, logout), Booking (booking_id, airline, flight_no, from_city, to_city, departure, arrival, price, status; methods: create_booking, cancel_booking), Passenger (name, age, gender, mobile, email), and Wallet (username, balance; methods: deduct, credit, get_balance). Figure 3.6 shows the complete class diagram as implemented in the system.

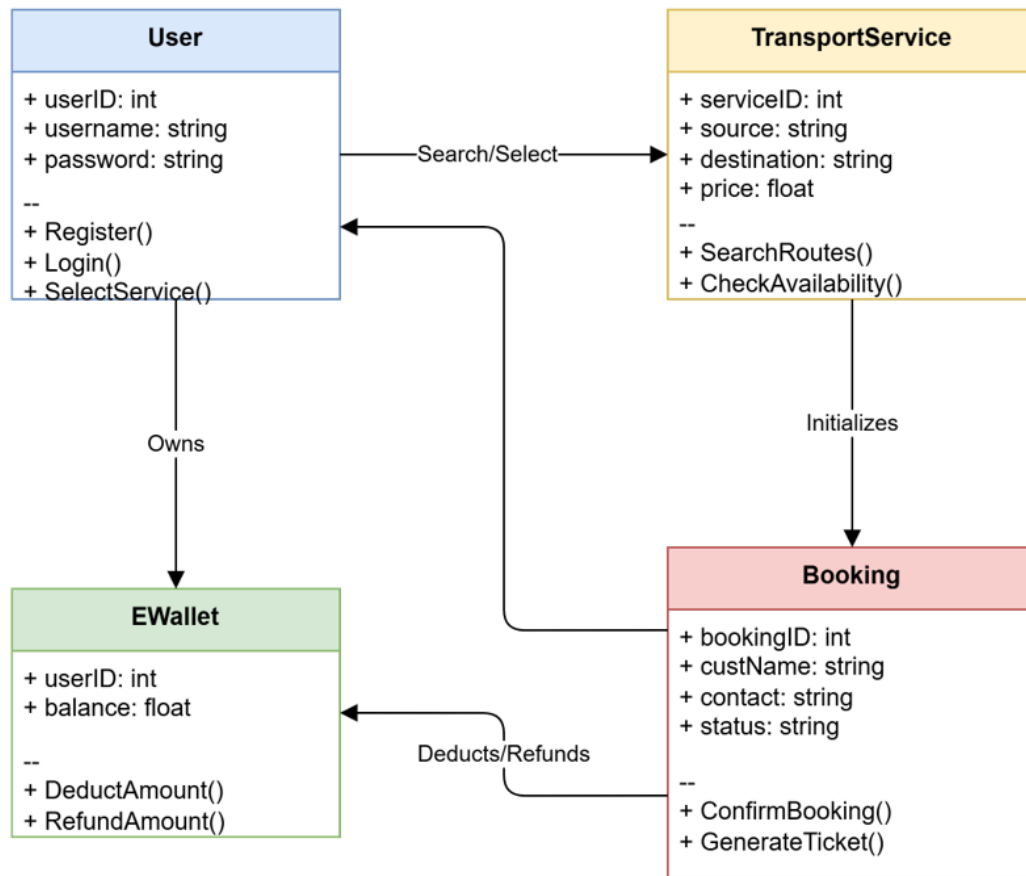


Figure 3.6: Class Diagram

3.4 Database Design

3.4.1 Logical Database Design

The logical schema consists of four relations. All functional dependencies are on the primary key; there are no transitive dependencies. The schema is normalized to avoid update, insertion, and deletion anomalies.

Table 3.1: Database Table: users

Column	Data Type	Constraint	Description
id	INT	PK, AUTO_INCREMENT	Unique user identifier
username	VARCHAR(100)	NOT NULL, UNIQUE	User login name
password	VARCHAR(255)	NOT NULL	User password (plain text)

Table 3.2: Database Table: bookings

Column	Data Type	Constraint	Description
id	INT	PK, AUTO_INCREMENT	Unique booking identifier
username	VARCHAR(100)	FK → users	Reference to booked user
airline	VARCHAR(100)	NOT NULL	Airline / operator name
flight_no	VARCHAR(50)	NOT NULL	Flight/train/bus number
from_city	VARCHAR(100)	NOT NULL	Departure location
to_city	VARCHAR(100)	NOT NULL	Destination location
departure	DATETIME	NOT NULL	Departure date and time
arrival	DATETIME	NOT NULL	Arrival date and time
price	DECIMAL(10,2)	NOT NULL	Ticket price including fees
booking_status	VARCHAR(50)	DEFAULT 'Confirmed'	Confirmed or Cancelled

refund_status	VARCHAR(50)	DEFAULT 'Not Cancelled'	Refund result after cancellation
booking_time	DATETIME	DEFAULT NOW()	Booking creation timestamp
payment_method	VARCHAR(50)	NOT NULL	Wallet / Card / UPI / Net Banking
payment_status	VARCHAR(50)	NOT NULL	Success or Failed

Table 3.3: Database Table: wallet

Column	Data Type	Constraint	Description
id	INT	PK, AUTO_INCREMENT	Unique wallet record identifier
username	VARCHAR(100)	FK → users, UNIQUE	Owner of the wallet
balance	DECIMAL(10,2)	DEFAULT 10000.00	Current wallet balance (Rs.)

Table 3.4: Database Table: passengers

Column	Data Type	Constraint	Description
id	INT	PK, AUTO_INCREMENT	Unique passenger record identifier
booking_id	INT	FK → bookings	Associated booking reference
name	VARCHAR(100)	NOT NULL	Passenger full name

age	INT	NOT NULL	Passenger age
gender	VARCHAR(10)	NOT NULL	Male / Female / Other
mobile	VARCHAR(20)	NOT NULL	Contact number with country code
email	VARCHAR(100)	NOT NULL	Passenger email address

Chapter 4

Implementation, Testing, and Maintenance

4.1 Introduction to Languages, IDEs, Tools and Technologies Used

TRAVEXA is built using a modern, industry-relevant web technology stack spanning backend, frontend, database, and API integration components. Table 4.1 provides a comprehensive summary of all languages, frameworks, libraries, and tools used in the implementation.

Table 4.1: Languages, IDEs, Tools and Technologies Used

Component	Technology	Version	Role in TRAVEXA
Backend Language	Python	3.9+	Core language for Flask backend
Backend Framework	Flask	2.x	Route handling, session management, template rendering
Database Connector	mysql-connector-python	8.x	MySQL connectivity and query execution
Flight Data API	Amadeus for Developers	SDK	Live flight search and airline data
Train Data API	RapidAPI IRCTC	v3	Real-time train schedule retrieval
HTTP Requests	requests	2.x	RapidAPI HTTP calls
Precision Arithmetic	Python decimal	Standard	Accurate price and refund calculations
Database	MySQL	8.0+	Persistent relational storage
Frontend	HTML5 & CSS3	—	Templating via Jinja2, page styling
IDE	VS Code	Latest	Python and HTML development

API Testing	Chrome Browser / Postman	—	Manual endpoint and API testing
Version Control	Git	2.x	Source code management
OS (Development)	Windows 10/11	—	Development and runtime environment

The backend follows a single-file Flask architecture (app.py) with Jinja2-based HTML templates in the templates/ directory. CSS styling is applied per-template. The Amadeus SDK is initialized with API credentials at application startup. MySQL connections are created per-request using the `get_db_connection()` helper function.

4.2 Testing Techniques and Test Plans

Testing followed a black-box approach using browser-based manual testing for all Flask route endpoints and manual API validation using browser developer tools. Table 4.2 presents the test cases executed to verify the functional correctness of the TRAVEXA system.

Table 4.2: Test Cases and Results

Test ID	Test Case Description	Input / Action	Expected Output	Status
TC-01	User Registration	New username and matching passwords submitted	User added to DB; session created; redirect to /home	Pass
TC-02	Duplicate Registration	Existing username submitted	Flash message: 'Already registered!'	Pass
TC-03	Login with valid credentials	Correct username and password	Session created; redirect to /home	Pass

TC-04	Login with invalid credentials	Wrong password submitted	Flash message: 'Invalid username or password'	Pass
TC-05	Flight Search (valid IATA codes)	DEL to BOM, future date	List of available flights displayed	Pass
TC-06	Flight Search (no results)	Invalid IATA codes	Message: 'No flights available for this route'	Pass
TC-07	Train Search	NDLS to BCT, future date	List of available trains from RapidAPI or error message	Pass
TC-08	Bus Search (high demand route)	Delhi to Manali	30-45 bus results displayed	Pass
TC-09	Bus Search (same city)	Delhi to Delhi	0 results; no match message	Pass
TC-10	Booking — Wallet payment	Select flight; wallet balance sufficient	Booking confirmed; wallet deducted	Pass
TC-11	Booking — Insufficient wallet	Select flight; wallet < total amount	Error: 'Insufficient Wallet Balance'	Pass
TC-12	Cancellation — within 24h of booking	Cancel booking made < 24 hours ago	Full refund credited to wallet	Pass
TC-13	Cancellation — within 2h of departure	Cancel booking with	No refund; booking marked Cancelled	Pass

		departure < 2h away		
TC-14	Cancellation — standard case	Cancel booking, > 24h from booking time, > 2h to departure	80% refund credited to wallet	Pass
TC-15	View bookings	Navigate to My Bookings	All bookings shown with status and refund info	Pass

4.3 Installation Instructions

4.3.1 Database Setup

- Install MySQL 8.0 or higher and start the MySQL service.
- Open a terminal or Command Prompt and log in: `mysql -u root -p`
- Create the database: `CREATE DATABASE travel_planner;`
- Select the database: `USE travel_planner;`
- Create the required tables: users, bookings, wallet, and passengers with the schema described in Chapter 3, Section 3.4.
- Verify setup: `SHOW TABLES;` — you should see four tables.

4.3.2 Backend Setup

- Ensure Python 3.9 or higher is installed. Verify: `python --version`
- Navigate to the project directory: `cd TravelPlanner_MakeMyTrip - Copy/travell`
- Activate the virtual environment: `venv\Scripts\activate` (Windows) or `source venv/bin/activate` (Linux/Mac)
- Install dependencies: `pip install flask mysql-connector-python amadeus requests`
- Edit `app.py` and update the MySQL password in `get_db_connection()` and the Amadeus API credentials.
- Start the Flask application: `python app.py`
- Access the application in a browser at: `http://localhost:5000`

4.4 End User Instructions

Registration: Open the application at <http://localhost:5000>. Click the Sign Up tab. Enter a unique username, password, and confirm password. Click Sign Up. The system creates an account and credits an initial E-Wallet balance of Rs. 10,000.

Booking a Flight: After login, click the Flights card on the Home page. Enter the source and destination IATA codes (e.g., DEL for Delhi, BOM for Mumbai) and the travel date. Click Search. Select a flight from the results and click Book. Enter passenger details and proceed to payment. Select E-Wallet, Card, UPI, or Net Banking and confirm. A booking confirmation is shown on the My Bookings page.

Booking a Train or Bus: The process is identical to flight booking. For trains, use station codes (e.g., NDLS for New Delhi). For buses, enter city names (e.g., Delhi, Mumbai).

Cancelling a Booking: Navigate to My Bookings. Click Cancel on the desired booking. The system automatically computes the refund and credits the E-Wallet. The booking status updates to Cancelled.

Viewing Wallet Balance: The current E-Wallet balance is displayed on the Home page and on the Wallet page accessible from the navigation.

Chapter 5

Results and Discussions

5.1 User Interface Representation

The TRAVEXA frontend is built using HTML5 templates rendered through Flask's Jinja2 templating engine and styled with custom CSS. The interface is organized into the following primary screens: Welcome / Login Screen, Home Screen, Search Results Screens (Flights, Trains, Buses), Passenger Entry Screen, Payment Screen, My Bookings Screen, and the E-Wallet Screen. Each screen presents a user-friendly interface designed to replicate the look and feel of commercial travel booking platforms such as MakeMyTrip.

5.2 Brief Description of Various Modules

The TRAVEXA system is implemented as a set of logically cohesive modules, each with a clearly defined responsibility:

- **Authentication Module:** Handles user registration (signup), credential validation (login), and session management (logout). All routes validate session presence before rendering protected pages.
- **Home Module:** Displays three service cards (Flights, Trains, Buses) and the current E-Wallet balance. Redirects unauthenticated users to the login page.
- **Flight Search Module:** Calls the Amadeus Flight Offers Search API with source IATA code, destination IATA code, and travel date. Filters results by departure time and displays airline name, flight number, departure/arrival times, duration, stops, price, and a promotional bank offer.
- **Train Search Module:** Calls the RapidAPI IRCTC endpoint with source and destination station codes. Processes the response to display train name, number, class, departure, arrival, duration, and price. Falls back gracefully if the API is unavailable.
- **Bus Search Module:** Uses the `get_buses_api_data()` function to generate realistic bus data based on route popularity. Displays operator, bus type, departure, arrival, duration, price, rating, and available seats.

- **Booking Module:** Collects passenger details (name, age, gender, mobile, email), computes the total fare, and processes payment via E-Wallet or simulated card/UPI/net banking. Inserts records into the bookings and passengers tables on success.
- **Cancellation & Refund Module:** Applies the three-rule refund algorithm and updates booking status. Credits the refund amount to the E-Wallet.
- **E-Wallet Module:** Maintains and displays the current wallet balance. Updates balance on booking (deduction) and cancellation (credit).

5.3 Snapshots of System with Brief Detail

The following figures show the key screens of the TRAVEXA system as they appear in the browser. Screenshots are to be added after project completion.

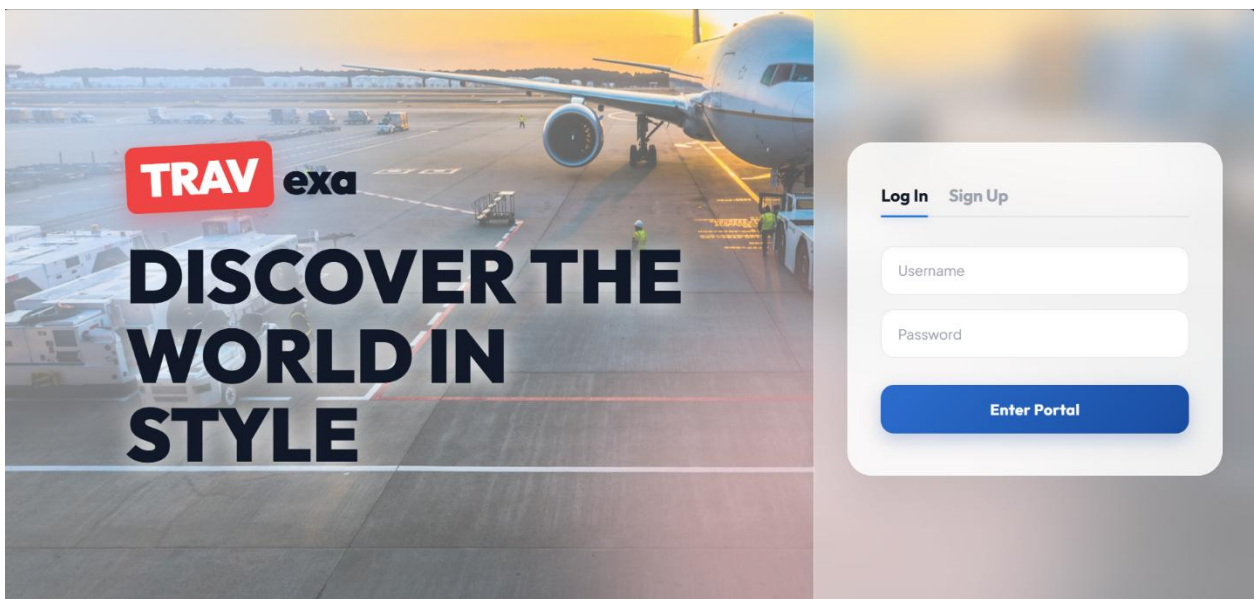


Figure 5.1: Login / Signup Screen — Users register or log in using the welcome page.

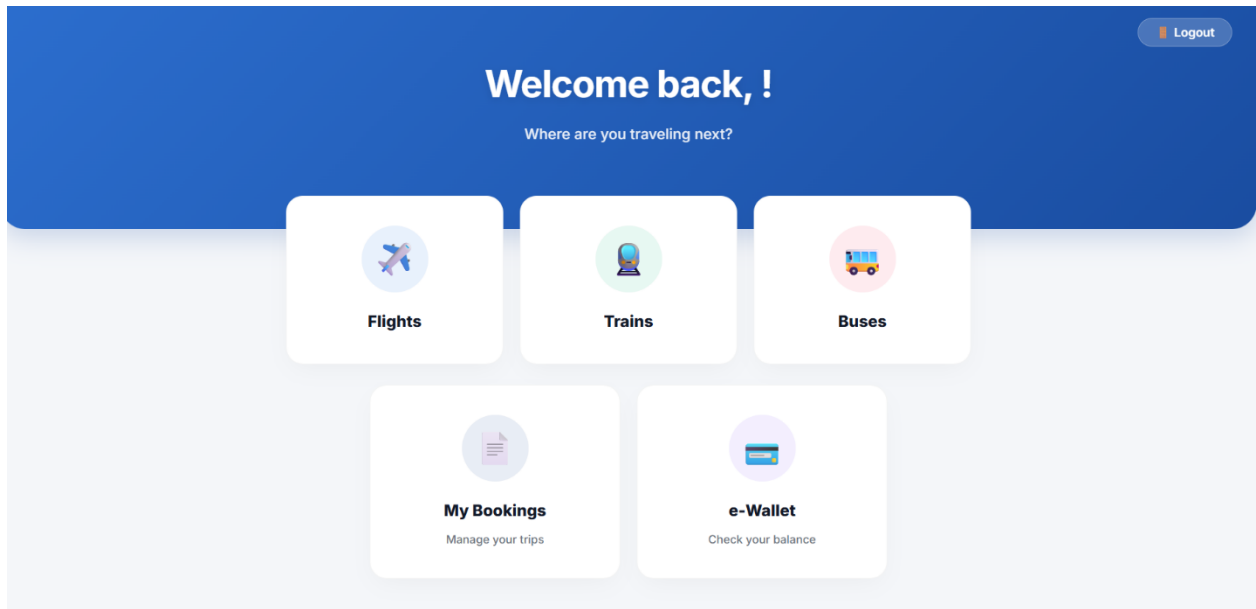


Figure 5.2: Home Screen — Three service cards for Flights, Trains, and Buses with E-Wallet balance.

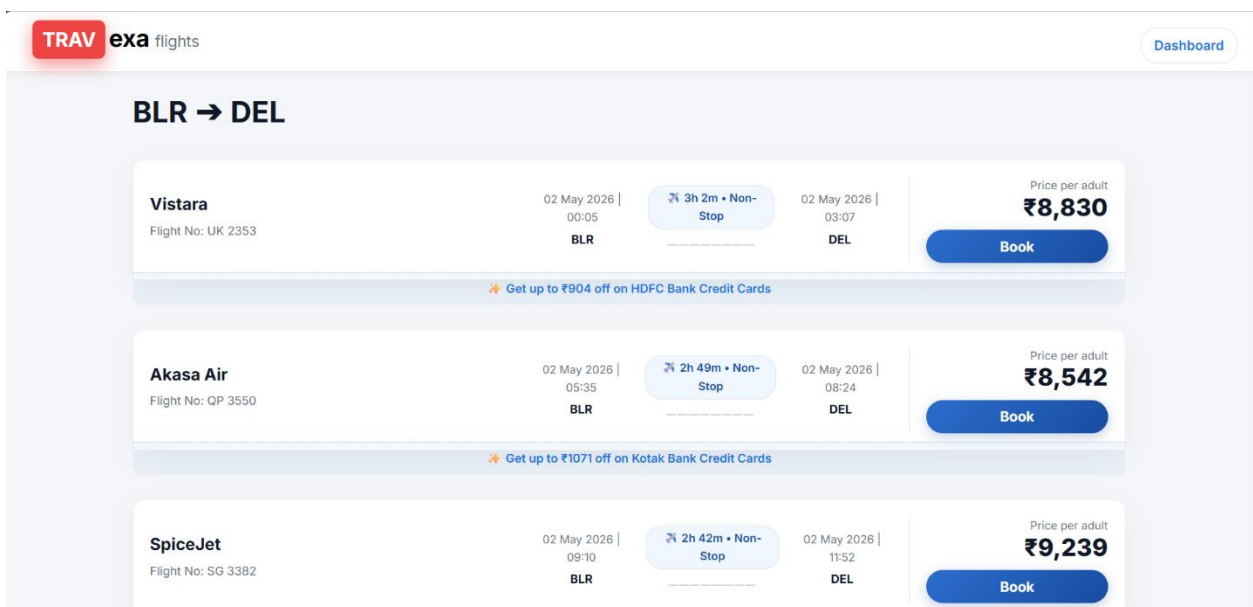


Figure 5.3: Flight Search Results — Flights displayed with airline, timings, price, and booking button.

TRAV

exa trains

Dashboard

UJN → BPL

NGP VANDEBHARAT

Train No: 20911

06:55

01 May 2026

UJN

2h 15m • 1A

09:10

01 May 2026

BPL

All inclusive fare

₹2,021

Book

Shipra Express

Train No: 22911

01:10

01 May 2026

UJN

2h 20m • 2A

03:30

01 May 2026

BPL

All inclusive fare

₹1,196

Book

Dr Ambedkar Nagar - Prayagraj Express

Train No: 14115

13:35

01 May 2026

UJN

2h 35m • 1A

16:10

01 May 2026

BPL

All inclusive fare

₹3,313

Book

Figure 5.4: Train Search Results — Trains displayed with operator, class, timings, and booking button.

TRAV

exa buses

Dashboard

Indore → Nasik

VRL Travels

Electric A/C Sleeper | UP92 GU 2405

06:30

29 Apr 2026

Indore

10h 0m • 19 seats left

16:30

29 Apr 2026

Nasik

Price per seat

₹2,287

Book

SRS Travels

Electric A/C Sleeper | UP40 NL 8216

09:00

29 Apr 2026

Indore

8h 30m • 19 seats left

17:30

29 Apr 2026

Nasik

Price per seat

₹1,557

Book

SRS Travels

BharatBenz A/C Sleeper (2+1) | UP89 OE 4930

18:00

29 Apr 2026

Indore

8h 15m • 23 seats left

02:15

30 Apr 2026

Nasik

Price per seat

₹1,942

Book

Figure 5.5: Bus Search Results — Buses displayed with operator, type, duration, price, and seat availability.

Enter Passenger Details

XYZ

45

+91 (IN

9284051845

xyz@gmail.com

Female

Proceed to Payment

← Back

Figure 5.6: Passenger Detail Entry — Form for entering passenger name, age, gender, mobile, and email.

Secure Checkout

Fare Summary

Base Fare (Adult x 1)

₹587.00

Convenience Fee

Included

₹350.00

Total Amount

₹937.00

* Non-refundable Convenience Fee

Select Payment Method:

☐  Credit / Debit Card

☐  UPI

☐  Net Banking

☐  Travexa e-Wallet

Pay ₹937

Figure 5.7: Payment Screen — Shows base fare, convenience fee, total, and payment method options.

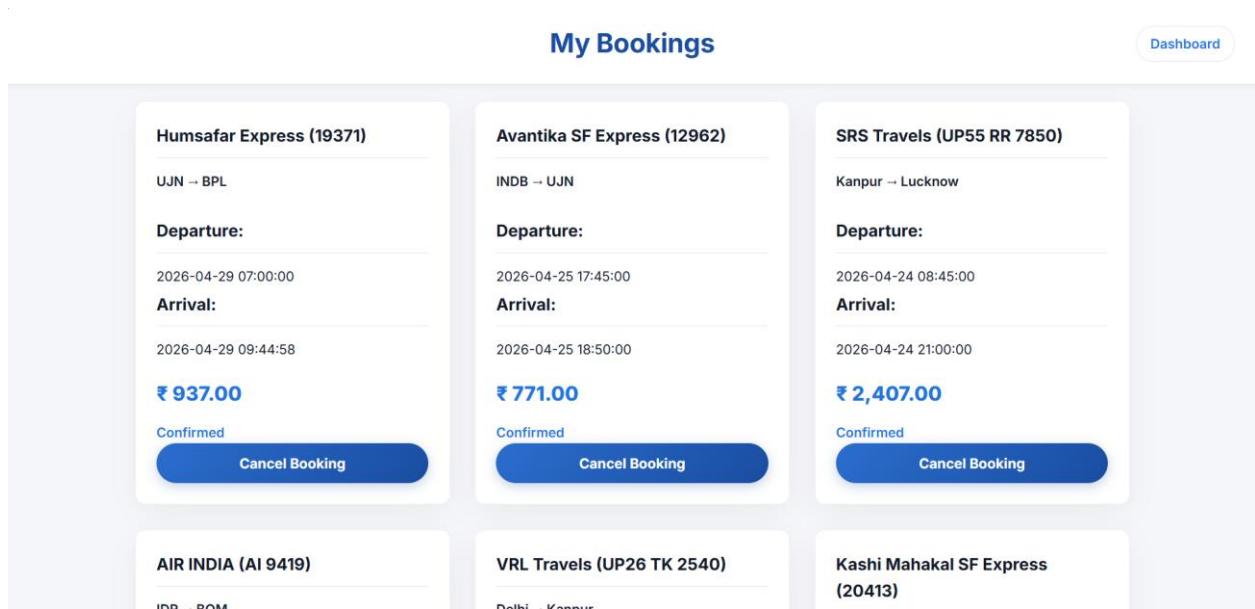


Figure 5.8: My Bookings Screen

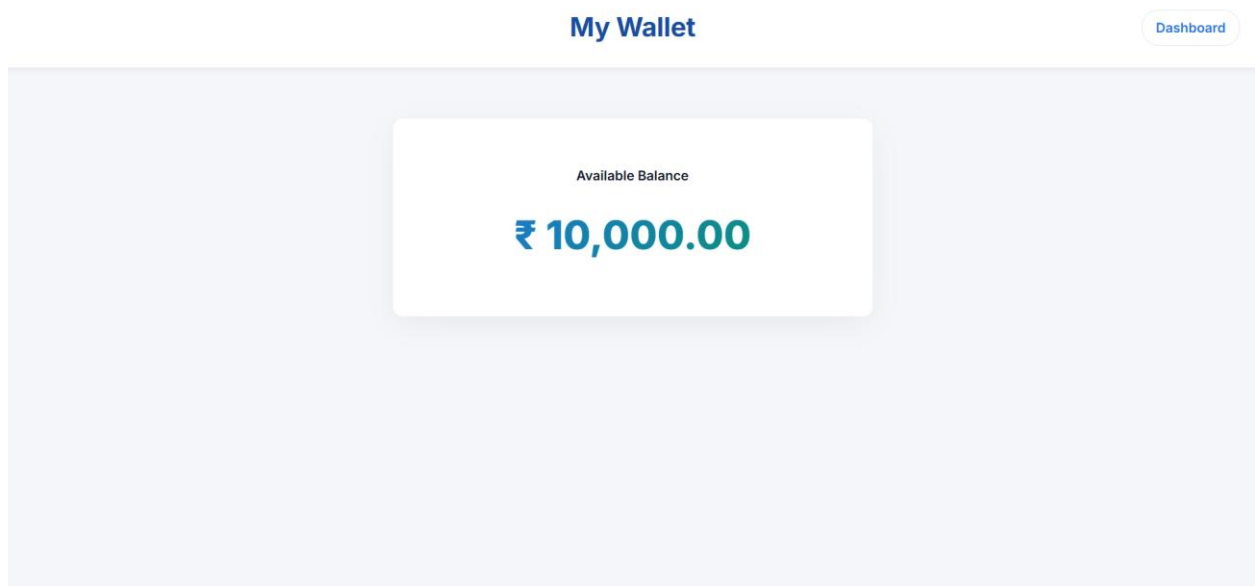


Figure 5.9: E-Wallet Screen — Displays current wallet balance

5.4 Back End Representation

The TRAVEXA backend is implemented as a single Flask application (app.py) running on the Flask development server at port 5000. The application follows a flat route-handler architecture with all business logic, database queries, and API calls contained within route functions. Key backend design decisions and their rationale are described below.

Decimal Arithmetic: All price and refund calculations use Python's Decimal type. Float and double types are avoided because IEEE 754 binary floating-point cannot represent many monetary fractions exactly. Decimal arithmetic ensures that refund amounts (e.g., 80% of Rs. 5,350) are computed correctly.

Session Management: Flask's session dictionary is used to maintain user login state across requests. The session also temporarily stores flight selection data (flight_data) and passenger details (passenger) during the multi-step booking process.

Pessimistic Locking: The wallet SELECT FOR UPDATE statement is used during payment and refund processing to prevent race conditions if concurrent requests attempt to modify the wallet balance simultaneously.

API Fallback: If the Amadeus or RapidAPI call fails, the system catches the exception and renders an appropriate error message without crashing. The bus module never makes an external API call, ensuring bus search always succeeds.

Table 5.1: Flask API Routes Summary

Method	Route	Description	Auth Required
GET	/ (welcome)	Landing page with login/signup forms	No
POST	/login	Validates credentials and creates session	No
POST	/signup	Registers new user and initializes wallet	No
GET	/logout	Clears session and redirects to login	Yes
GET	/home	Displays home page with service cards	Yes
GET	/flights-search	Renders flight search form	Yes
POST	/search-flights	Calls Amadeus API, returns flight results	Yes
GET	/trains	Renders train search form	Yes

POST	/search-trains	Calls RapidAPI, returns train results	Yes
GET	/buses	Renders bus search form	Yes
POST	/search-buses	Generates bus mock data, returns results	Yes
POST	/enter-passenger	Stores flight selection, renders passenger form	Yes
POST	/payment	Computes total fare, renders payment screen	Yes
POST	/process-payment	Processes payment, inserts booking record	Yes
GET	/my-bookings	Displays all bookings for logged-in user	Yes
POST	/cancel-booking	Applies refund logic, updates booking status	Yes
GET	/wallet	Displays E-Wallet balance	Yes
GET	/api/v1/trains	Internal REST API for train data	No
GET	/api/v1/buses	Internal REST API for bus data	No

5.5 Snapshots of Database Tables with Brief Description

The four MySQL database tables store all persistent data for the TRAVEXA system. The users table is the root entity. The wallet table is linked one-to-one with the users table. The bookings table is the primary fact table with one row per confirmed booking. The passengers table stores traveller details for each booking. Database snapshots from MySQL Workbench are to be inserted below.

	id	username	password
▶	1	admin	pass
	2	adminn	pass
	3	admin1	pass1
	4	Aarya Saraf	Pa5s!w0rd23
	5	testuser	Password123!
	6	unique_user_2024	Pass123!
★	NULL	NULL	NULL

	id	booking_id	name	age	gender	mobile	email	state
▶	1	1	Priya Sharma	34	Female	NULL	NULL	NULL
	2	2	Arpita	22	Female	NULL	NULL	NULL
	3	3	Arpita	22	Female	NULL	NULL	NULL
	4	4	Arpita	22	Female	NULL	NULL	NULL
	5	5	Arpita	22	Female	NULL	NULL	NULL
	6	6	Priya Sharma	75	Female	NULL	NULL	NULL
	7	7	Priya Sharma	75	Female	NULL	NULL	NULL
	8	8	Priya Sharma	75	Female	NULL	NULL	NULL
	9	9	Arpita	43	Female	NULL	NULL	NULL
	10	10	Arpita	43	Female	NULL	NULL	NULL
	11	11	Arpita	43	Female	NULL	NULL	NULL
	12	12	Arpita	64	Female	NULL	NULL	NULL
	13	13	Arpita	56	Female	NULL	NULL	NULL
	14	14	Akshita	67	Female	NULL	NULL	NULL
	15	15	Arpita	23	Female	NULL	NULL	NULL
	16	16	Arpita	23	Female	NULL	NULL	NULL
	17	17	Arpita	23	Female	NULL	NULL	NULL
	18	18	Arpita	23	Female	NULL	NULL	NULL
	19	19	Arpita	23	Female	+91 ...	xyz...	Mad...
	20	20	Arpita	23	Female	+91 ...	xyz...	Mad...
	21	21	Arpita	23	Female	+91 ...	xyz...	Mad...
	22	22	RUHI	27	Female	3896...	xyz...	Mad...
	23	23	RUHI	27	Female	3896...	xyz...	Mad...
	24	24	RUHI	27	Female	3896...	xyz...	Mad...

Figure 5.10: MySQL Workbench showing tables of the travel_planner database.

Chapter 6

Summary and Conclusions

This report presented the complete design, implementation, testing, and evaluation of TRAVEXA — Smart Travel Planner, a unified web-based travel booking system developed using Python Flask and MySQL. The following summary describes the key outcomes achieved across all phases of the project.

In the requirements phase, a comprehensive specification was produced identifying the Traveller user role, fifteen functional requirements covering authentication through refund processing, software and hardware dependencies, performance targets, and system constraints.

In the design phase, the Booking Algorithm and Refund Computation Algorithm were formally defined. A four-table normalized MySQL schema was designed. Data Flow Diagrams (Level 0 and Level 1), Use Case Diagram, Activity Diagram, Flowchart, Class Diagram, and ER Diagram were produced to document the system's functional and structural characteristics.

In the implementation phase, the Flask backend was built with nineteen route handlers covering all system operations. The Amadeus API was integrated for live flight data, the RapidAPI IRCTC endpoint for real train schedules, and a realistic mock engine for bus data. The frontend was implemented using Jinja2 HTML templates and CSS, providing a MakeMyTrip-inspired user interface.

In the testing phase, fifteen test cases were executed covering normal operations, boundary conditions (same-city bus search, insufficient wallet), API error handling, and all three refund scenarios. All fifteen test cases passed successfully.

In conclusion, TRAVEXA successfully achieves all stated objectives. It provides a seamless, unified travel planning and booking experience that eliminates the need to visit multiple websites for different transport services. The system demonstrates the effective integration of web development, database management, third-party API consumption, session management, and digital refund processing in a cohesive, academically sound application.

Chapter 7

Future Scope

While TRAVEXA delivers a fully functional and tested travel booking system, several enhancements are identified as future development directions:

- **Payment Gateway Integration:** Replacing the simulated payment system with a real gateway such as Razorpay or PayPal to enable actual digital payments.
- **Password Hashing:** Implementing bcrypt or SHA-256 password hashing to improve authentication security.
- **Mobile Responsive UI:** Optimizing the system for smartphones and tablets using responsive CSS frameworks such as Bootstrap.

Appendix

Appendix I — Flask Application Configuration

The following key configuration settings are used in app.py:

- `app.secret_key = 'supersecretkey'` — Flask secret key for session encryption.
- `MySQL host='localhost', user='root', password='<YOUR_PASSWORD>', database='travel_planner'` — Database connection parameters.
- `Amadeus client_id` and `client_secret` — Credentials for Amadeus API access (to be replaced with valid developer credentials).
- `RAPID_API_KEY` — RapidAPI key for the IRCTC1 train data endpoint.
- `app.run(debug=True, threaded=True)` — Starts the Flask development server with threading enabled.

Appendix II — Database Initialization SQL

The database is initialized by executing SQL CREATE TABLE statements for the four tables: users, bookings, wallet, and passengers. The bookings table requires foreign key references to the users table. The passengers table requires a foreign key reference to the bookings table. An initial wallet balance of Rs. 10,000 is inserted for each new user via the signup route after account creation.

Bibliography

- [1] D. Buhalis and R. Law, "Progress in Information Technology and Tourism Management," Tourism Management, 2008.

- [2] M. Grinberg, Flask Web Development, 2nd ed. Sebastopol: O'Reilly Media, 2018.

- [3] Flask Official Documentation, Pallets Projects, 2024. [Online]. Available: <https://flask.palletsprojects.com> [Accessed: Feb. 2026].

- [4] MySQL 8.0 Reference Manual, Oracle Corporation, 2024. [Online]. Available: <https://dev.mysql.com> [Accessed: Feb. 2026].

- [5] Amadeus for Developers API Documentation. [Online]. Available: <https://developers.amadeus.com> [Accessed: Mar. 2026].

- [6] RapidAPI — IRCTC API Documentation. [Online]. Available: <https://rapidapi.com/IRCTC/api/irctc1> [Accessed: Mar. 2026].

- [7] Jinja2 Template Engine Documentation, Pallets Projects. [Online]. Available: <https://jinja.palletsprojects.com> [Accessed: Mar. 2026].

- [8] R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner's Approach, 9th ed. New York: McGraw-Hill Education, 2020.

- [9] Python Software Foundation, Python 3.9 Documentation. [Online]. Available: <https://docs.python.org/3> [Accessed: Feb. 2026].

- [10] Tutorialspoint, "Online Travel Booking System." [Online]. Available: <https://www.tutorialspoint.com> [Accessed: Jan. 2026].